

Lecture 3:

UTF-8 Text Representation

CSE 29: Systems Programming and Software Tools

Aaron Schulman (Shalev)



Introducing: UTF-8 Encoding

- UTF-8 (Unicode Transformation Format - 8-bit) is an extension to ASCII
- That gives us more characters than what's in ASCII
- It is a universal standard used across the internet and the globe
- In the modern day, all the text data we use is in this standard
 - ◆ 99% of the text we interact with uses UTF-8 encoding (websites, text, emails, etc.)
- It uses variable-length encoding that's backward-compatible with ASCII
 - ◆ More about this on the next slide

Problem:

How do we encode a char as multiple bytes?



Dec	Hx	Oct	Html	Chr	Dec	Hx	Oct	Html	Chr	Dec	Hx	Oct	Html	Chr	Dec	Hx	Oct	Html	Chr
64	40	100	@	@	96	60	140	`	`	81	51	121	Q	Q	113	71	161	q	q
65	41	101	A	A	97	61	141	a	a	82	52	122	R	R	114	72	162	r	r
66	42	102	B	B	98	62	142	b	b	83	53	123	S	S	115	73	163	s	s
67	43	103	C	C	99	63	143	c	c	84	54	124	T	T	116	74	164	t	t
68	44	104	D	D	100	64	144	d	d	85	55	125	U	U	117	75	165	u	u
69	45	105	E	E	101	65	145	e	e	86	56	126	V	V	118	76	166	v	v
70	46	106	F	F	102	66	146	f	f	87	57	127	W	W	119	77	167	w	w
71	47	107	G	G	103	67	147	g	g	88	58	130	X	X	120	78	170	x	x
72	48	110	H	H	104	68	150	h	h	89	59	131	Y	Y	121	79	171	y	y
73	49	111	I	I	105	69	151	i	i	90	5A	132	Z	Z	122	7A	172	z	z
74	4A	112	J	J	106	6A	152	j	j	91	5B	133	[	[	123	7B	173	{	{
75	4B	113	K	K	107	6B	153	k	k	92	5C	134	\	\	124	7C	174	|	
76	4C	114	L	L	108	6C	154	l	l	93	5D	135	]	]	125	7D	175	}	}
77	4D	115	M	M	109	6D	155	m	m	94	5E	136	^	^	126	7E	176	~	~
78	4E	116	N	N	110	6E	156	n	n	95	5F	137	_	_	127	7F	177		DEL
79	4F	117	O	O	111	6F	157	o	o										
80	50	120	P	P	112	70	160	p	p										

Source  www.LookupTables.com

Solution: Bit flags!

- **New Terminology... “Code Point”:**
 - ◆ A code point is an integer representing a character (e.g., 65 codepoint of 'A')
- **Normal ASCII Code Point:** Highest order bit of byte is **0xxxxxxx**
- **Multi-byte Code Point:** Highest order bit of byte is a flag **10xxxxxx**
- **First bits of first byte of code point dictates code point length:**
 - ◆ **110xxxxx = 2 bytes**
 - ◆ **1110xxxx = 3 bytes**
 - ◆ **11110xxx = 4 bytes**
- **Bytes in the middle (called continue bytes) start with:**
 - ◆ **10xxxxxx**

What do UTF-8 code points look like in memory?

byte 1

byte 2

byte 3

byte 4

1-byte ASCII = **0xxxxxxx**

2-byte UTF-8 = **110xxxxx 10yyyyyy**

3-byte UTF-8 = **1110xxxx 10yyyyyy 10zzzzzz**

4-byte UTF-8 = **11110xxx 10yyyyyy 10zzzzzz 10wwwwww**

Codepoints

- `U+#####` is the way we write the codepoint of a character to other humans
 - ◆ ASCII 'A' is 41 which is codepoint U+0029
 - ◆ Crab is codepoint 129408 which is codepoint U+1F980
- Hexadecimal is a compact way of writing binary data (i.e., integers)
 - ◆ 16 options per digit (0-F) rather than 10 in decimal (0-9) and 2 in binary (0-1)
 - ◆ Compact way of representing bits because each digit represents 4 bits in memory ($2^4=16$)
 - ◆ We distinguish hexadecimal numbers by prepending them with 0x.....

Code point construction from multi-byte

□ é =

0b110	00011	0b10	101001
-------	-------	------	--------

2 bytes

continue

000011101001 = 233 = 0xE9 = U+000E9

The Code Point

Example of 4-byte character

- Unicode Character: 🦀
- Codepoint (in hex): U+1F980
- Binary representation: 11110000 10011111 10100110 10000000
- 1 Starting Byte (11110xxx)
- 3 Continuation Bytes (10xxxxxx)
- To get the codepoint, we use the non-prefix bits of the binary

11110**000** 10**011111** 10**100110** 10**000000** -> **000****011111****100110****000000** -> 0**1F980**

Analyzing codepoints requires accessing individual bits in C data types



It is possible to control/access individual bits of Integer data types in C

- ◆ `int*_t` datatypes all can have bit-wise operations

There are special mathematical operators in C for doing operations on bits

- ◆ **&** - AND
- ◆ **|** - OR
- ◆ **^** - XOR
- ◆ **>>** - Shift Right
- ◆ **<<** - Shift Left

Boolean operators work on **all bits of a variable (e.g., 8 bits in a char)**

How bit operations work: OR

```
char a = 2; // 00000010 in binary
```

```
char b = 5; // 00000101 in binary
```

```
// OR each bit of the two integers together
```

```
char a_or_b = a | b;
```

```
// 00000010
```

```
// | 00000101
```

```
// -----
```

```
// 00000111
```

```
printf("%d\n", a_or_b); // What will the output be?
```

Bit operations to do uppercase()

- ▣ Subtraction is needed to go from lowercase to uppercase (-32)
- ▣ Uppercase and lowercase ASCII are 32 apart... hmmmmmmm
- ▣ Can I implement uppercase without subtract?
- ▣ **Demo** ...

How bit operations work: AND

```
char a = 3; // 00000011
```

```
char b = 5; // 00000101
```

```
// AND each bit of the two integers together
```

```
char a_and_b = a & b;
```

```
// 00000011
```

```
// & 00000101
```

```
// -----
```

```
// 00000001 // Lets you select bits you want to inspect
```

```
printf("%d\n", a_and_b); // What will the output be?
```

Masking: use case for bitwise AND (&)

□ Masking:

- ◆ Selecting specific individual bits out of a binary representation of a number
- ◆ Example:
 - » `first_four_bits( 0b01010101 ) = ?`
 - » `last_four_bits( 0b00000011 ) = ?`
 - » `first_four_bits( last_four_bits(____) );`

How do we implement masking?

```
char msb( char c ) {  
    return c & 0b10000000;  
}
```

```
char lsb( char c ) {  
    return c & 0b00000001;  
}
```

- ◆ Using the bitwise AND operator **&** we can select specific bit positions to view
- ◆ The bits selected come from the pattern of 1's that the variable is **&'ed** with

Demo: Masking used in practice!

- **Big Idea:**

- ◆ Using masking, we can inspect a character to see if it's UTF-8 multibyte codepoint or not